

Basics4AI — Programmer's Guide

Version 1.0 — 2026-07-11. Companion to docs/solution-architecture-document.md — read that first for the high-level system picture. This document is for a developer who needs to run, extend, or operate the platform.

1. Local Development Setup

Prerequisites

- The b4ai_v0 conda environment (Python 3.10, matching the Dockerfile's python:3.10-slim). This is the known-working local environment — a separate env, basics4aiv1, exists and throws unexplained errors; avoid it.
- A .env file in the repo root (gitignored, never committed). At minimum:
 - RID_SALT — any string ≥32 characters, used to derive research IDs consistently.
 - ANTHROPIC_API_KEY, OPENAI_API_KEY, GEMINI_API_KEY, GROQ_API_KEY — only required if you're touching the LLM-assisted analysis features (ITA/DTA pipelines); other features work without them.
 - QUIZ_MODE — "random" or "research" (controls MCQ question-bank selection).

Running the app

With the conda environment activated:

```
streamlit run streamlit_app/app.py
```

That's it — no separate database initialization step is required. streamlit_app/app.py calls `init_db()`, `init_auth_db()`, `init_login_attempts_table()`, and `register_all_modules()` automatically on first request. Fresh SQLite files will be created in the repo root if none exist.

The R/rpy2 caveat — you probably don't need `launch.py`

`launch.py` exists solely to initialize R on the main thread before Streamlit starts, working around an rpy2/Ctrl-C interaction bug on Windows. **It is not needed for normal development.** The actual IRT (Item Response Theory) analysis feature no longer uses R at all — it was migrated to the pure-Python girth library (`core/analytics/irt/irt_runner.py`), and the R integration is vestigial: no R is installed in the Docker image, rpy2 isn't even a listed dependency, and the app degrades gracefully without it. Only reach for `launch.py` if you're specifically debugging something in the old R code path (unlikely to ever be necessary going forward).

Testing

There is currently **no automated test suite** in the working tree, and **no CI job runs tests on push/PR** — the only GitHub Actions workflow is the nightly production-restart cron (§9). A `tests/` directory with ~21 manual verification scripts (organized phase0–phase4, one per analytics module) existed earlier in the project’s history and was removed; git history preserves them if a future developer wants to resurrect the pattern (`git log --oneline` → find the commit that deleted `tests/`, then `git show <parent-commit>:tests/phase0/test_binary_mcq.py` to recover an example). These were plain Python scripts (`python tests/phase0/test_binary_mcq.py`), not pytest-collected tests, despite pytest being listed in `requirements.txt`.

What does exist: **`smoke_test.py`** (repo root, gitignored, local-only) — run `python smoke_test.py` before manual testing. It checks: both databases are reachable, `canonical_loader` runs without error, `research.db`’s cache table exists, the `rid` migration columns are present (and reports coverage), and SQLite is actually in WAL mode with an adequate busy timeout. Exit code 0 means all checks passed.

Recommendation for future work: given there’s no automated regression coverage today, any nontrivial change should be manually verified against a copy of real data (not an empty database) before merging — this is the discipline already established for the pilot’s change-safety workflow (§3).

2. Environment Variables Reference

Variable	Purpose	Read by
DB_TYPE	sqlite (default) or postgres — the Postgres path is scaffolding only, not functional (psycopg2 isn’t installed)	core/db_utils.py
SQLITE_PATH	Override for responses.db’s location	core/db_utils.py, core/admin/research_service.py, core/admin/diagnostics_service.py, core/admin/system_service.py
USERS_DB_PATH	Override for users.db’s location	auth/login_security.py, auth/user_manager.py, core/admin/user_service.py, core/admin/diagnostics_service.py,

Variable	Purpose	Read by
		core/admin/system_service.py
DATA_DIR	Base directory for SQLite files (used on Railway, pointing at the mounted volume)	core/db_utils.py
RESEARCH_DB_PATH	Intended location of research.db — not actually read by any app code , only by smoke_test.py. See the Known Limitations appendix in the Solution Architecture Document.	smoke_test.py only
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	PostgreSQL connection params — unused until the DB_TYPE=postgres path is made functional	core/db_utils.py
LOGIN_MAX_ATTEMPTS	Failed-login threshold before lockout (default 3)	auth/login_security.py
LOGIN_LOCKOUT_MINUTES	Lockout duration in minutes (default 5)	auth/login_security.py
LOGIN_WINDOW_MINUTES	Rolling window for counting failures (default 5)	auth/login_security.py
BACKUP_INTERVAL_HOURS	Auto-backup scheduler interval (default 24)	core/admin/system_service.py
RID_SALT	HMAC salt for deriving research IDs from usernames — see §4's schema notes and the post-pilot roadmap for the current state of RID adoption	migrations/backfill_rids.py, auth/user_manager.get_user_rid call sites
RAILWAY_ENVIRONMENT	Set automatically by Railway — used to skip loading a local .env file in production	streamlit_app/app.py
QUIZ_MODE	"random" or "research" — controls MCQ question selection	utils/quiz_mode.py, module render functions
QUIZ_QUESTION_IDS_MODULE_{1-7}	Fixed per-module question ID lists, used only when	utils/quiz_mode.py

Variable	Purpose	Read by
	QUIZ_MODE=research	
ANTHROPIC_API_KEY, GEMINI_API_KEY, OPENAI_API_KEY, GROQ_API_KEY	LLM provider credentials for the qualitative analysis pipelines	core/analytics/llm/llm_clients.py
R_HOME, RPY2_CFFI_MODE	Vestigial — R integration path, not needed in normal use (see §1)	streamlit_app/app.py, core/analytics/r_utils.py, launch.py
RAILWAY_TOKEN	Railway CLI auth for the nightly restart — a GitHub Actions secret, never read by application code	.github/workflows/scheduled-restart.yml

3. Deployment Workflow — the Standard Change-Safety Procedure

This is the procedure used throughout the pilot-readiness work and should be the default for any future change, however small it looks:

1. **Build and test locally** against a realistic copy of data (not an empty database) — `smoke_test.py` first, then manual verification of the actual feature in a browser.
2. **Push to a disposable test git branch.** Never commit directly to main.
3. **Deploy to Railway's test environment** and re-verify there. Note: test's Railway service is *not* separately wired to auto-deploy from the test branch — both production and test environments' services deploy from whatever's pushed to origin/main. To get code onto the test *environment* without touching main, use a manual deploy: `railway environment test, railway service basics4ai-staging`, then `railway up --ci --environment test --service basics4ai-staging`. (The Railway CLI can report a spurious “request error / operation timed out” on this command even when the deploy actually succeeded — always confirm with `railway status --json` rather than trusting the CLI's exit code.)
4. **Only after both local and live-test-environment verification, merge test → main.** Do a dry-run conflict check first (`git merge-tree $(git merge-base main test) main test`) before the real merge.
5. **Push to origin/main.** This automatically triggers a production redeploy — there is no separate approval gate on Railway's side, so this push *is* the production deploy. Treat it accordingly: take a fresh production database backup immediately beforehand (see §7's backup tooling), and review the full diff one more time before pushing.
6. **Verify production after deploy:** check `railway status --json` for SUCCESS, hit the health endpoint (`https://basics4ai-staging-production.up.railway.app/_stcore/health` should return 200), check railway

logs for a clean startup with no tracebacks, and — for any change affecting module registration or similarly session-scoped initialization — open the production URL in a real browser (not just curl) since that logic only runs on an actual Streamlit websocket session, not a plain HTTP request.

Both Railway environments (production and test) share one service definition and only differ in their attached volume and environment variables — this is why a push to main deploys to both simultaneously, and why steps 4-5 above deserve real caution.

4. Database Schema Reference

Full table-by-table detail. See the Solution Architecture Document §5 for the narrative version; this is the exhaustive reference.

`responses.db`

Table	Key columns	Notes
responses	user_id, rid, instrument_name, question_id, response_value, submitted_at	One row per question answered. rid column exists but is not populated for any existing row (0/758 as of this writing) — see the post-pilot roadmap for the RID-adoption gap.
survey_scores	user_id, rid, survey_key, score, calculated_at	Unique on (user_id, survey_key) — upserted, not appended.
assessment_scores	user_id, rid, assessment_code, score, calculated_at	Unique on (user_id, assessment_code) — upserted, not appended.
completions	user_id, rid, module_id, instrument_key, completed_at	This is the unlock source of truth. Unique on (user_id, module_id, instrument_key); writes use INSERT OR IGNORE, making completion marking idempotent.
transcripts	participant_id, uploaded_by, ...	Interview transcript storage for qualitative analysis. participant_id is who the transcript is about; uploaded_by is the admin who uploaded it — these

Table	Key columns	Notes
		must never be conflated (the former is research-subject data, the latter is audit provenance).
ita_runs / ita_results	run_id (UUID)	Inductive Thematic Analysis pipeline outputs. ita_results has no direct user identifier — it links back to student data purely through run_id.
dta_runs / dta_results / dta_lo_results	run_id (UUID)	Deductive Thematic Analysis pipeline outputs — same linkage pattern as ITA.
cpi_runs / cpi_qual_scores / cpi_summary	—	Competency Progression Index engine outputs — see §8.

users.db

Table	Key columns	Notes
users	id, username, password (hashed), role (CHECK constraint: student/teacher/admin only), cohort_id, status, is_super_admin, rid	super_admin is not a stored role value — it's role='admin' + is_super_admin=1, synthesized into the string "super_admin" at read time by get_user_role().
login_attempts	id, username, ip, timestamp, success	Powers the lockout mechanism.
cohorts	cohort_id (PK)	Named participant groupings.
admin_logs	id, timestamp, admin_user, action, details	Every admin action is logged here — see §7.
restore_log	id, restored_at, admin_user, backup_timestamp, pre/post row counts, verified	Created lazily on first use of the restore feature.

research.db

One table: llm_result_cache (input_hash PK, provider, run_mode, n_texts, result_json, created_at) — a content-hash-keyed cache to avoid re-paying for identical LLM analysis calls.

Connection-handling consistency — read before adding a new DB-touching file

core/db_utils.py's get_connection() is the **only** connection path that correctly sets PRAGMA journal_mode=WAL, synchronous=NORMAL, and busy_timeout=5000. Several existing files (auth/login_security.py, auth/user_manager.py, core/admin/user_service.py, core/admin/system_service.py, core/admin/audit_logger.py, core/admin/diagnostics_service.py) open their own raw sqlite3.connect() without these pragmas — this is a known inconsistency (documented in the Solution Architecture Document §8.3), not something to imitate. **Any new code that opens a database connection should import and use core.db_utils.get_connection()** rather than calling sqlite3.connect() directly — this is exactly the fix already applied to core/admin/data_service.py, which should be the template to follow.

5. Module Lifecycle

Adding, hiding, re-enabling, or replacing module content is documented as its own standalone procedure in docs/module-lifecycle-guide.md — treat that as the canonical reference rather than duplicating it here. In brief: every module's status field (in its modules/definitions/moduleN_definition.py file) is the single switch that controls whether it appears to students; everything else (registry discovery, unlock sequencing, Post-Course gating) derives from that automatically.

6. How the Core Engines Fit Together — a Full Submission Walkthrough

This traces exactly what happens when a student answers a Module 1 MCQ question, as a concrete reference for extending or debugging the submission path.

1. **Entry:** modules/module_1.py's registry-invoked render(username) fetches completion state, then dispatches to render_content_mcq(assessment_def, assessment_key, username) for the MCQ instrument.
2. **Question loading & UI:** questions are loaded (mode-aware — QUIZ_MODE=random vs research), rendered as an st.form with one st.radio per question.

3. **Client-side scoring:** on submit, the module itself — not `scoring_engine.py` — computes the score by comparing selected answers against each question's answer field, then calls:

```
submit_instrument(  
    user_id=username, module_id=MODULE_ID,  
    instrument_key=assessment_key, responses=letter_responses, score=score  
)
```

4. **Inside `core/submission_engine.py:submit_instrument()`**, in order:

1. Resolve rid via `auth.user_manager.get_user_rid(user_id)` if not already provided (silently None on failure).
 2. Validate the instrument is actually required by this module (`get_required_instruments()` from `core/progress_engine.py`) — raises `ValueError` otherwise.
 3. Open a connection via `core.db_utils.get_connection()`.
 4. **Write one INSERT INTO responses per question answered.**
 5. Determine instrument type ("survey" vs "assessment") via `get_instrument_type()`.
 6. **Auto-scoring branch:** only triggers if `score` is None *and* the instrument is a survey — loads the relevant `*_scoring.yaml` from `streamlit_app/surveys/` and calls `core.scoring_engine.compute_score()`. Not triggered for MCQs, since the module already computed and passed a score explicitly.
 7. **Upsert the score** into `assessment_scores` (or `survey_scores`), keyed on `(user_id, assessment_code) / (user_id, survey_key)` — repeated submissions overwrite, they don't accumulate duplicate rows.
 8. Commit and close.
 9. **Mark completion:** `core.db_utils.mark_instrument_complete()` does an idempotent INSERT OR IGNORE INTO completions — this is the row the unlock logic (`core.progress_engine.is_module_unlocked()`) reads on the student's next dashboard view.
5. **Back in the UI:** the module marks the instrument complete in `st.session_state`, shows a success message, and reruns — at which point the Student Dashboard's live unlock check picks up the new completions row and unlocks the next module if this was the last required instrument.

A subtlety worth knowing: `scoring_engine.compute_score()` supports MCQ-style scoring too (an "Option A" binary mode matching `correct_answers: {Q1: "A", ...}` in a scoring YAML, including graceful handling of randomized question subsets), but Module 1's actual MCQ handler doesn't use it — it scores client-side instead. If you're adding a new module, either pattern is valid; just be consistent about which one you use so the scoring logic isn't duplicated in two places for the same instrument.

Reflections behave differently: they pass `score=None` and are classified as "assessment" type, but since the auto-scoring branch only fires for "survey" type, **no score row is ever written for a reflection** — only the raw responses rows and the completions marker. This is intentional; reflections are qualitative and scored later (if at all) by the LLM analysis pipeline, not by `submission_engine.py`.

7. Admin Operations Reference

A practical, per-feature guide to what each Admin Dashboard section actually does on the backend. All admin actions are logged to `admin_logs` via `core/admin/audit_logger.py`.

Dashboard tab	Section	Backend function	What it does
User Management	Create/Delete User, Reset Password	<code>auth/user_manager.py</code>	Standard account lifecycle. <code>reset_password</code> generates a cryptographically random password (<code>generate_password()</code>), not a predictable one.
User Management	Login Lockout Management	<code>auth/login_security.clear_lockout()</code> , <code>set_lockout_enabled()</code>	<code>clear_lockout(username)</code> deletes only that user's failed attempts within the current rolling window — cannot affect other users or successful logins. The pilot-mode toggle is process-local and resets to "enabled" on every restart (including the nightly cron restart) by design — it's meant for temporary use during a single in-person session, not a persistent setting.
Data Management	Reset One	<code>core/admin/data_service.reset_user_i</code>	Deletes are scoped

Dashboard tab	Section	Backend function	What it does
	Instrument for One Student	<code>nstrument(), count_user_instrument_rows()</code>	by both <code>user_id</code> AND the instrument column across all four core tables — cannot touch another student or another instrument for the same student. Always preview counts first via <code>count_user_instrument_rows()</code> before calling the delete.
System Operations	Download Databases Now	<code>core/admin/system_service.backup_databases() + st.download_button</code>	Runs a fresh backup, zips both <code>.db</code> files, and serves them directly to the admin's browser — the only backup path that actually leaves Railway's infrastructure. Use this before any risky operation, and periodically regardless.
System Operations	Auto-Backup Status	<code>core/admin/system_service._catch_up_if_stale()</code>	Runs automatically on every app start; if the most recent backup is older than <code>BACKUP_INTERVAL_HOURS</code> , it backs up immediately rather than waiting for the next scheduled interval — this closes a gap where frequent restarts (e.g. the nightly cron) could otherwise starve the backup

Dashboard tab	Section	Backend function	What it does
			schedule indefinitely.
System Operations	Restore Databases	<code>core/admin/system_service.restore_databases()</code>	Verifies row counts before and after, logs to <code>restore_log</code> . Built but not yet actually tested end-to-end as of this writing — verify this works before relying on it in a real incident.
Diagnostics	Audit Log	<code>core/admin/audit_logger.get_all_logs_df()</code>	Exportable as CSV — every admin action, who did it, and when.

8. Analytics Pipeline Reference

The Teacher Dashboard’s six sections are backed by `core/analytics/`, organized by analysis type: `descriptive/`, `inferential/`, `irt/`, `cpi/`, `llm/`, and `datasets/` (which assembles the shared “canonical dataset” every other engine consumes).

What’s implemented and live, briefly (see the Solution Architecture Document for the general shape, and the post-pilot roadmap memory for a detailed audit if you’re picking this work back up):

- **Descriptive statistics** — participant summaries, assessment score aggregation, survey construct means.
- **Inferential statistics** — paired comparisons, including a fully-implemented Bland-Altman method-agreement analysis (`core/analytics/inferential/inferential_tests.py:run_bland_altman()`), cited and rendered in the dashboard.
- **IRT (Item Response Theory)** — pure-Python via the `girth` library (no R dependency despite some stale docstrings still mentioning `mirt/rpy2` — see the Solution Architecture Document §8.6).
- **Construct reliability** — CR, AVE, and McDonald’s ω , converting IRT discrimination parameters to factor loadings — fully implemented in `core/analytics/irt/reliability_analysis.py` and wired into the dashboard.
- **CPI (Competency Progression Index)** — `core/analytics/cpi/cpi_engine.py` contains **two different combining formulas**: the original two-component $CPI+ =$

$w1 * \text{CPI_quant} + w2 * \text{CPI_qual}$ (this is the one actually used and persisted by the live Teacher Dashboard tab), and a newer three-component $\text{CPI_outcome} + \text{CPI_process} + \text{CPI_qual}$ model (including Hake’s normalized gain for pre/post delta scores) that exists as working code but is **not wired into the live tab** — it’s only reached as a fallback inside the PDF report generator, and even there the report’s own methodology text still describes the old formula. **This is a known, real inconsistency** — if you’re asked to “finish” CPI+, this mismatch is the first thing to resolve, not a new feature to build.

- **LLM analysis (ITA/DTA)** — inductive and deductive thematic analysis via LLM provider APIs, with result caching in `research.db`.

What’s explicitly out of scope for this document (tracked separately, not yet built or only partially built): the Hybrid Qual+Quant integration tab, an expanded Reports tab (inter-rater reliability reports, keyword-driven reports, freeform researcher Q&A), and full RID-based de-identification of the analytics pipeline (today, `canonical_loader.py` and everything downstream reads by username, not by research ID, despite RID columns existing in the schema). See the post-pilot roadmap for detail — these are deliberately sequenced after the pilot, not part of the current system.

9. Known Issues & Operational Runbook

Streamlit websocket/session-leak (upstream bug)

There is an unresolved, maintainer-acknowledged bug in Streamlit itself ([streamlit/streamlit#8901](#)) involving accumulated memory/session state on long-running processes. No application-level fix exists (or is expected upstream soon), so the mitigation is purely operational: `.github/workflows/scheduled-restart.yml` triggers a Railway redeploy of the production service every night at 07:00 UTC via `railway redeploy --service <id> --yes`, authenticated with a `RAILWAY_TOKEN` secret. This was empirically verified to be scoped specifically to the production environment (not test) by observing production’s deployment ID change on a manual trigger while test’s stayed identical — worth re-verifying if the token is ever regenerated, since Railway service IDs are identical across environments and only the token’s own scoping determines which environment actually gets restarted.

Operational consequence: because this restart happens nightly, any process-local (non-persisted) state resets every 24 hours. Two places in the codebase already account for this by design: - The auto-backup scheduler’s catch-up check (§7) — prevents backups from silently stopping due to frequent restarts. - The admin lockout-pause toggle (§7) — intentionally resets to “enabled” (the safe default) on every restart rather than staying paused indefinitely.

If you add new process-local state in the future, consider whether it needs the same “restart-safe” treatment.

Deployment quirk: spurious CLI timeout

`railway up/railway redeploy` can report `request error... operation timed out` even when the deployment actually succeeded on Railway's backend — this is a CLI-side polling/streaming timeout, not a real failure. Always confirm actual status via `railway status --json` rather than trusting the CLI's exit code or printed error.

Windows-only quirks (local development)

- Windows consoles default to a legacy codepage (cp1252) that can't encode the emoji used throughout the app's `status print()` statements — `streamlit_app/app.py` forces UTF-8 stdout/stderr at the very top of the file to prevent local (non-Docker) runs from crashing on the first such print.
- `MSYS_NO_PATHCONV=1` is required when running `railway ssh/railway volume` commands from Git Bash that reference Unix-style paths (e.g. `/app/data/...`) — otherwise Git Bash auto-mangles them into Windows paths and the command fails.
- `railway ssh` does not reliably support piped stdin (hangs indefinitely, confirmed) — for pushing file content to a remote volume, base64-encode in ~20,000-character chunks passed as command *arguments*, not piped stdin.

10. Known Limitations

See `docs/solution-architecture-document.md` §8 for the full, current list (hardcoded super-admin default credentials, unsalted password hashing, WAL-mode inconsistency, the `research.db/RESEARCH_DB_PATH` configuration mismatch, the non-functional Postgres scaffolding, and several instances of dead/vestigial code) — not duplicated here to avoid the two documents drifting out of sync. Treat the Solution Architecture Document as the source of truth for known limitations; this Programmer's Guide should only describe *how the system works*, not *what's wrong with it*.